

Training Data & Eval Playbook

Owner: Marv **Purpose:** Working reference for training-data acquisition and eval-methodology decisions. Private notes; feed into Ronin sync + ADR drafts. **Chosen approach (current):** Independent external dataset + manual review + scripting output adaptation for eval process. **Deadline stance:** Extended for stronger research quality. Speed over quality was rejected; deep, defensible eval takes precedence.

1. The problem in one paragraph

BAYMAX is eval-driven — its research value depends on whether the eval actually measures the model’s ability to reason about tool selection on **novel unseen queries**, not whether it can pattern-match to a narrow distribution shape. Training data must therefore be **semantically and structurally different** from eval, and the eval itself must be broad, diverse, and stratified enough that a strong score genuinely means something. Getting this wrong invalidates the entire fine-tune experiment regardless of what number the eval produces.

2. Three failure modes — must be understood before choosing an approach

These get conflated all the time. They are distinct problems with distinct fixes.

Failure mode	What it looks like	Standard fix
1. Strict contamination	Same example literally appears in both train and eval sets	N-gram overlap check (script); trivial
2. In-distribution overfitting	Eval scenarios are structurally / stylistically similar to training. Model pattern-matches to distribution shape without understanding	Different authors for train vs eval; diverse eval; embedding-similarity checks
3. Poor generalization	Model can’t reason about tool selection on genuinely novel query shapes it wasn’t trained on	Compositional held-out splits; explicit out-of-distribution (OOD) eval tier; adversarial subsets

BAYMAX’s project docs currently address #1 explicitly and #2 implicitly. #3 is not addressed at all. That is the load-bearing gap in the current methodology.

3. Eval design shortfalls (BAYMAX’s current state)

Assessment from an outside-critic perspective on the eval as it currently stands (50 scenarios, single author, no held-out splits):

3.1 Volume is thin

50 scenarios has ~10-15% variance on a proportion metric. A 5-point score difference between backends could be pure noise. For any research claim about relative model quality to survive statistical scrutiny:

Sample size	Statistical noise band	Can detect
50	~10-15%	Only large differences (>15 points)
200	~6-8%	Moderate differences (>10 points)
500	~4-5%	Small differences (>5 points)

Ronin’s roadmap targets 200 by Phase 3, which is minimum viable but late.

3.2 No stated diversity structure

All 50 scenarios are hand-authored by one person (Ronin). Without a structural audit (bucketing by tool combination, phrasing type, intent shape, error mode), effective diversity might be much lower than 50 suggests — possibly 5-10 recurring patterns dressed as 50 variants.

3.3 No compositional held-out split

Standard practice for testing generalization: train on scenarios involving tools A+B and A+C individually, then evaluate on scenarios combining B+C — combinations never seen in training. Tests whether the model learned tool semantics vs memorized specific combinations. Not planned in current docs.

3.4 No OOD tier

Rigorous eval design includes an explicit out-of-distribution subset — deliberately novel phrasings, adversarial edge cases, unfamiliar tool combinations — reported separately from the in-distribution score. Absent here.

3.5 Single-author bias not addressed

Same person writing both training data and eval scenarios bakes distributional overfitting in even without literal contamination. LLM-generation partially mitigates this but doesn't eliminate it.

3.6 No pre-registered failure criterion

Docs don't state what result counts as “the fine-tune didn't work.” Without this, any outcome can be rationalized as success, which reviewers see through immediately.

4. Contamination and bias issues — project-specific concerns

4.1 Verification cost problem

The naive picture is “run n-gram overlap check, done.” That catches strict contamination — verbatim reuse. It does not catch semantic paraphrase or structural pattern overlap, which is 90% of what actually creates in-distribution overfitting.

Cost-vs-reliability trade for detection methods:

Approach	Catches	Cost	Reliability
N-gram overlap (5-gram)	Verbatim reuse	Free (script)	Only #1 (strict contamination)
Embedding cosine similarity	Semantic paraphrase	Cheap (script + embeddings)	~60-70% — misses structural similarity in different wording
Structural fingerprinting	Pattern overlap	Human taxonomy + automated matching	Better; only as good as the taxonomy
Full human pairwise review	Everything	50 eval × 2000 train = 100K pairs @ 5s each = 140 hrs	Best possible, unaffordable

Approach	Catches	Cost	Reliability
Sample-based statistical review (100 pairs)	Estimated overlap rate	1-2 hrs	Statistical only — no per-pair guarantee

There is no cheap+reliable option. Sample-based statistical review is what BAYMAX can afford; it produces a defensible “estimated overlap rate = X” but not a per-scenario guarantee.

4.2 LLM-generation homogeneity problem

Real-world result, not opinion: LLMs mode-collapse under bulk generation.

- Ask GPT-4o for 2000 scenarios → get ~50-200 base patterns with shallow variations
- Coverage biased toward “happy path” — happy-agent-succeeds scenarios overrepresented; refusal, ambiguity, adversarial inputs, edge cases underrepresented
- BAYMAX baseline is already weak on refusal (21%) and clarification (55%); LLM-gen data will inherit those blind spots, not fix them
- Style regression: everything sounds like GPT-4o; fine-tune inherits the style

Documented in the synthetic-data literature: - Wang et al. 2022 (Self-Instruct) — showed bulk LLM generation produces heavy mode collapse; invented iterative bootstrapping to break out of it - Xu et al. 2023 (Evol-Instruct / WizardLM) — explicitly designed to force LLMs OUT of default generation modes via evolutionary prompt operators - GPT-4 tech report — dedicates entire section to contamination checks

4.3 Author-split protection

When the SAME PERSON writes both train and eval, instincts about edge cases and phrasing carry over. Even without literal contamination, training over-covers exactly the cases eval tests.

Current split (Ronin owns eval): partial protection. **Full protection:** Marv MUST NOT look at Ronin’s eval scenarios while designing or generating training data. Not even a sample. Discipline is what makes the separation real. Training data generation should be based on tool schema and functional spec, NOT on example eval scenarios.

5. Dataset acquisition options — full menu

Every technique below is a real-world approach with published precedent. Each has trade-offs. This is the menu to reason from, not a ranking.

5.1 Core acquisition strategies

Technique	What it does	Reference	Honest limits
A. Pure external public dataset	Use ToolBench / API-Bank / ToolAlpaca / Gorilla as-is	HuggingFace	Wrong tool namespaces; wrong response format; doesn't teach BAYMAX-specific patterns (verification, <code>_contains</code> , refusal)
B. Custom hand-written by Marv/Ronin	You write ~1000-2000 scenarios per your tool schema	—	Highest quality per example; matches your schema; but highest author bias — the killer for research
C. LLM-generated custom	GPT-4o generates ~2000 scenarios per your spec	Standard practice	Matches schema; mode-collapses; needs review; realistic case scores 35-45%
D. Hybrid (public + custom)	Bootstrap on public, then domain-adapt with custom	Industry standard for domain adaptation	Most complex; needs alignment between phases; the “correct” answer if you have time
E. LLM-generated with output adaptation	Use external data as-is; script adapts model output to match eval format	← CHOSEN APPROACH	See §6 for full analysis

5.2 Data generation / augmentation techniques

Techniques that layer on top of any core strategy above:

Technique	What it does	Reference	Honest limits
Self-Instruct	Seed with 175 human tasks, iteratively bootstrap thousands more via LLM	Wang et al. 2022; Stanford Alpaca 2023	Distributional bias remains; Alpaca models showed poor OOD reasoning
Evol-Instruct	LLM evolves simple instructions along 5 dimensions (add constraints, deepen reasoning, complicate, concretize, breadth)	Xu et al. 2023 (WizardLM)	Increases complexity distribution but same style regression
Rejection sampling	Generate 10x-20x what you need, aggressively filter by quality/diversity heuristics, keep top N	Standard in RLHF (InstructGPT, Llama 2)	Filter design is another judgment problem
Constitutional AI critique loop	LLM critiques its own generations against explicit rubric, rewrites failing ones	Bai et al. 2022 (Anthropic)	Introduces rubric's own biases; helps quality more than diversity
Programmatic template generation	Slot-filling from vocabulary lists (“{verb} a {time} meeting with {name} about {topic}”)	Classic NLP; dialogue-system training	Trades naturalness for control; guarantees structural coverage; sounds robotic
Multi-model generation	Generate a third from GPT-4o, third from Claude, third from Gemini	Common industry practice	Reduces per-model bias; doesn't eliminate LLM-mode-collapse; 3x API cost
Back-translation paraphrasing	Translate scenarios EN → ZH → EN via translation model	Standard in MT data augmentation	Adds phrase diversity, not structural diversity; translation artifacts

5.3 Advanced / high-quality techniques

Technique	What it does	Reference	Honest limits
DynaBench / adversarial collection	Author scenarios specifically designed to break current models; iterate as models improve	Kiela et al. 2021 (Meta AI Research)	Requires a working model to attack; slow; highest data quality
Existing personal-assistant datasets	Closer-domain data: MultiWOZ, SGD (Schema-Guided Dialogue), TaskBench, ToolQA, MINT (multi-turn tool interactions)	Multiple published on HuggingFace	Each has own schema mismatch; needs adapter layer
Two-tier eval (public + hidden)	Release 30 scenarios publicly for iteration; keep 20 hidden for final numbers	Standard for public benchmarks (Kaggle, GLUE test set)	Doesn't help YOUR fine-tune's train/eval separation, but useful for publication reproducibility
Delegated authorship	Someone in Marv's network (who understands schema but hasn't seen eval) writes training seeds	Free with the right person	Requires trust + skill; contribution scope bounded
Crowdsourcing (MTurk / Prolific)	Pay real humans \$5-20 per scenario	Standard in academic NLP	$2000 \times \$5 = \$10K$. Out of budget for BAYMAX; listed for completeness
Real user logging	Deploy agent, collect real user interactions as training/eval data	Gold standard for production	Requires shipped product + users; not viable at BAYMAX's current phase

6. Chosen approach — External dataset + review + output adaptation (analysis)

6.1 What this actually means

- **Training data:** use a pre-existing external dataset (ToolBench, API-Bank, or similar) without modifying it

- **Review:** sample-review the external data for quality; skip full curation
- **Output adaptation script:** write a translation layer that converts whatever tool-call format the fine-tuned model emits (matching the external dataset’s format) into BAYMAX’s `AgentResponse` shape before the eval scorer sees it

6.2 Why this approach was chosen

- **Zero author-bias in training data** — data was written by people who have never seen BAYMAX’s eval scenarios. Cleanest possible separation.
- **No LLM-mode-collapse concern** — data comes from human authors (or human-labeled at scale), not from bulk LLM generation
- **Time-efficient** — no data authoring or curation burden on Marv/Ronin
- **Defensible research narrative** — “trained on public data, evaluated on our held-out scenarios” is a clean methodology story
- **Output adapter is the innovation** — reframes the “wrong tool namespace” problem from a data problem into an engineering problem

6.3 What can go wrong

Expected outcome: fine-tune underperforms gpt-4o-mini baseline by 5-15 points, because: - Training data teaches the model tool-calling in a foreign namespace/pattern - Output adapter has to fix mismatched calls at inference time - Adapter is brittle at boundaries (e.g., ToolBench emits `calendar.create(...)` but BAYMAX expects `calendar.create_event(...)` — how does the adapter know?)

Specific risk categories: - **Adapter coverage** — every tool ToolBench doesn’t have but BAYMAX needs is a hole (e.g., `notion.mark_task_done` may not exist in ToolBench; adapter has no ToolBench signal to translate from) - **Argument shape mismatch** — external data may use different argument names/types than BAYMAX - **Refusal behavior mismatch** — ToolBench doesn’t teach “refuse this request” the way BAYMAX needs - **Clarification behavior mismatch** — same as above - **Format drift** — model may emit ToolBench-style JSON that the adapter has to parse, and parsing failures compound

6.4 Mitigations to layer on top

Given the chosen approach, the following techniques from §5 become more valuable, not less:

1. **Adversarial hole-finding** — before training, enumerate every tool in BAYMAX’s contract; check which ones have direct analogs in the external dataset; the gap list is the adapter’s TODO

2. **Programmatic augmentation for gaps** — for tools/behaviors that don’t exist in the external dataset (refusal, clarification, `_contains` patterns), template-generate a small supplementary training set (~100-200 examples). Small enough that homogeneity concerns are manageable
3. **Multi-tier eval reporting:**
 - **In-distribution tier:** scenarios similar in shape to what the external dataset covers → baseline capability signal
 - **BAYMAX-specific tier:** scenarios using BAYMAX-unique behaviors (refusal, verification) → adapter effectiveness signal
 - **OOD tier:** novel scenarios not resembling anything in external training data → true generalization signal
4. **Adapter test suite** — the adapter script itself needs unit tests. Every ToolBench→BAYMAX translation is a test case. Adapter bugs will look like model failures otherwise
5. **Statistical power** — even with this approach, eval size needs to reach 200+ for the results to be interpretable
6. **Author-split protection** — Marv writes/reviews training-adjacent code (adapter); Ronin owns eval scenarios; neither looks at the other’s work in ways that would leak patterns

6.5 Reframed research claim

The strongest defensible narrative with this approach is NOT “small fine-tune beats frontier model” but:

“We characterize the ceiling of transfer learning from public tool-use datasets to a domain-specific agent contract. Our approach uses [external dataset X] for training, an output-adaptation layer to bridge namespace differences, and a held-out BAYMAX-specific eval to measure transfer effectiveness. Results indicate [X%] task success — [Y points below baseline]. Failure-mode analysis reveals [specific patterns], suggesting [where domain-specific data would be needed].”

That is publishable as an ablation / negative-space characterization study. It is genuinely useful because MANY teams face this exact tradeoff (use public data cheaply vs curate domain data expensively) and there is not enough written about what the transfer ceiling actually is.

7. Comparison framing options (research narrative choices)

Regardless of approach, the research claim needs framing. Head-to-head “fine-tune beats frontier” is nearly impossible to defend cleanly. Alternatives:

Framing	Comparison	Defensibility
A. Fine-tune vs its own base	Qwen 1.5B base (~15-25%) vs Qwen 1.5B + fine-tune (target ~40-50%)	Clean; measures fine-tuning delta
B. Fine-tune vs same-tier baseline	Qwen 1.5B + BAYMAX fine-tune vs a published 1-2B tool-use fine-tune (e.g., Phi-2 or TinyLlama on ToolBench)	Defensible; comparing like-for-like at parameter tier
C. Cost-normalized frontier vs small	gpt-4o-mini cost (\$0.00012/scenario) vs local Qwen (~\$0) — “achieves N% quality at 0% of the cost”	Practical; useful for deployment story
D. Transfer-learning ceiling (matches chosen approach)	External-data-trained fine-tune’s transfer to domain eval; characterize the gap	Publishable as ablation/negative-space study
E. Fine tune vs frontier head-to-head	Rarely defensible for small models	Reviewers will dismiss

Recommendation: **run A + D as parallel headline claims**. Both are defensible. Neither requires the fine-tune to win.

8. Open questions for Ronin sync (2026-07-29)

1. **Which external dataset?** — ToolBench (largest, general), API-Bank (curated, narrower), or another? Ronin’s SFT experience should drive this.
2. **Adapter script ownership** — Marv’s side (runtime) or Ronin’s side (post-processing before scorer)?
3. **Diversity audit of eval** — 20-min task; classify the 50 scenarios into structural buckets. If they fall into <5 buckets, eval effective-diversity is a bigger concern than volume.
4. **Held-out OOD subset** — reserve 10-20 of the target-200 scenarios as explicit OOD; author them separately from any training data workflow.
5. **Compositional split plan** — can we structure the tool combinations in training vs eval to test compositional generalization?

6. **Eval volume timeline** — 200 scenarios is minimum for statistical power. When?
 7. **Statistical noise reporting** — mean $\pm$ CI on aggregate metrics, not just point values.
 8. **Author-split discipline** — Marv commits to not looking at eval scenarios during any training-data-adjacent work. Ronin commits to not looking at the adapter’s translation table during eval-scenario authoring.
 9. **Pre-registered failure criterion** — what result counts as “fine-tune didn’t work”? Agree BEFORE running the experiment.
 10. **Comparison framing** — which of §7’s options becomes the headline claim?
-

9. Bottom line

- The chosen approach (external dataset + review + output adaptation) is a defensible research posture that avoids the worst LLM-mode-collapse and author-bias problems.
 - It comes with predictable costs (fine-tune underperforms baseline; adapter is brittle at boundaries) which the project narrative must be prepared to own.
 - Weak eval design + strong fine-tune result = numbers no one takes seriously.
 - Strong eval design + weak fine-tune result = a defensible negative finding, which IS a research contribution.
 - The eval methodology is more load-bearing than the fine-tune outcome.
 - 20-40 hours of focused work over the extended timeline turns “unknown quality” into “quantified quality with documented gaps” — the real cost of doing this properly.
-

10. References

- Wang, Y. et al. 2022. **Self-Instruct: Aligning Language Models with Self-Generated Instructions**.
- Xu, C. et al. 2023. **WizardLM / Evol-Instruct: Empowering Large Language Models to Follow Complex Instructions**.
- Bai, Y. et al. 2022. **Constitutional AI: Harmlessness from AI Feedback** (Anthropic).
- Kiela, D. et al. 2021. **DynaBench: Rethinking Benchmarking in NLP** (Meta AI Research).
- Zhou, C. et al. 2023. **LIMA: Less Is More for Alignment** (Meta) — 1000 quality examples beat 50K medium-quality.
- Taori, R. et al. 2023. **Stanford Alpaca** — Self-Instruct in practice.
- OpenAI 2023. **GPT-4 Technical Report** — contamination section, methodology.

- Qin, Y. et al. 2023. **ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs** (ToolBench).
- Li, M. et al. 2023. **API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs**.
- Patil, S. et al. 2023. **Gorilla: Large Language Model Connected with Massive APIs**.
- Nexusflow 2023. **NexusRaven** function-calling model.

Related memory: [[baymax-citations]] in Claude’s memory system holds a growing pool of these references with brief characterizations.